

# Running Hermes Standalone (Overnight Mode)

**Created:** 2026-04-11 **Why this exists:** The 2026-04-10 overnight run on `Crispi-app-evr` died at 17:04:53 because Claude Code's `Bash(run_in_background: true)` does NOT detach the spawned process from its parent. When the Claude Code session ended, SIGHUP cascaded down and killed hermes mid-call. Hermes has no SIGHUP handler in `run_agent.py`, so the session JSON was 3 minutes stale. **This guide is the safe way to launch hermes so it survives session close.**

---

## TL;DR — Launch a Standalone Run

Run this from a normal terminal (NOT inside Claude Code's bash tool):

```
cd /media/bob/C1/AI_Projects/Crispi-app

LOG_FILE=~/.hermes/logs/overnight-evr-$(date +%Y%m%d_%H%M%S).log

setuid -f bash -c "
  cd /media/bob/C1/AI_Projects/Crispi-app && \
  hermes chat -s dev-team/vibe-loop --yolo -m qwen3.5:cloud \
    -q 'Brownfield: Continue Crispi-app-evr (landing page redesign). Story spec
already exists at docs/stories/evr-beta-signup-redesign.md. Beads issue Crispi-app-
evr is open. Start at story-specs (it will auto-detect the existing spec and skip to
tdd). Read AGENTS.md for project rules. Run all phases through quinn-review and e2e-
validation. Halt only on Phase 12 (deploy) for secrets.' \
    > $LOG_FILE 2>&1
"

echo "Hermes launched. Log: $LOG_FILE"
echo "PID: $(pgrep -f 'hermes chat')"
```

## Why each piece matters

| Flag / pattern                                        | Purpose                                                                                                                                                                                                      |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>setsid -f</code>                                | Creates a NEW session group, fully detached from controlling terminal. Immune to SIGHUP from parent shell death. The <code>-f</code> flag forks so the command continues in the background without blocking. |
| <code>bash -c "..."</code>                            | Wraps the command in a shell so <code>cd</code> and the redirect work cleanly inside <code>setsid</code> .                                                                                                   |
| <code>cd /media/bob/C1/AI_Projects/Crispi-app</code>  | Hermes uses <code>cwd</code> to find <code>_bmad-output/</code> , <code>.beads/</code> , <code>AGENTS.md</code> . Must be in the project root.                                                               |
| <code>hermes chat -s dev-team/vibe-loop --yolo</code> | Invokes the vibe-loop skill in YOLO mode (no human gates between phases).                                                                                                                                    |
| <code>-m qwen3.5:cloud</code>                         | Cloud model. 256K context, ~12-60s/call latency. Switch to <code>hermes3:8b</code> once P40s are installed.                                                                                                  |
| <code>-q '...'</code>                                 | One-shot query. Hermes completes the pipeline and exits.                                                                                                                                                     |
| <code>&gt; \$LOG_FILE 2&gt;&amp;1</code>              | Captures stdout AND stderr to the timestamped log. Critical for post-mortem if it dies.                                                                                                                      |

## Critical: do NOT launch this from inside Claude Code's bash tool

Claude Code's `Bash(run_in_background: true)` parameter spawns the command as a CHILD of Claude Code's bash runner, not detached. When Claude Code closes (or the session times out, or the 10-minute Bash timeout fires), the child gets SIGHUP and dies. **`setsid -f` is the only reliable way to detach from a Claude Code parent.**

If you absolutely must launch from inside Claude Code, the same `setsid -f` invocation works there too — but verify detachment with `ps -ef | grep hermes` after closing Claude Code. The hermes process should still be running with PPID=1 (init/systemd).

## Interactive Chat Modes (NOT autonomous pipeline)

Sometimes you want to **just talk to hermes** — ask a question, debug something, brainstorm, or use the dev-team skills interactively without kicking off the full vibe-loop pipeline. These modes are for that. They run in the foreground in your terminal, you can `Ctrl+C` to exit, and your conversation is auto-persisted to `~/hermes/sessions/`.

**No `setsid` needed for interactive modes.** You're sitting in front of the terminal, you want to see the output, you're not going overnight. The detachment trick is only for unattended overnight runs.

### Bare interactive chat (no skill loaded)

```
cd /media/bob/C1/AI_Projects/Crispi-app hermes chat
```

## New Interactive Chat:

Click on the Hermes icon in panel or open in Linux Menu. This will open a headless Hermes-Vibe-Tool Chat. Use this to kick around ideas then you could ask hermes to creat new projects. Based on your existing layout,

/media/bob/C1/AI\_Projects/{project-name}/ is where your AI projects live (that's where hermes-dev-team is). Tell hermes to put new ones there and you stay organized.

When to start a fresh Hermes Chat session on the new project.

Eventuall you should. After vibe-loop has scaffolded the new project, it may create a project-local config.yaml with its own system instructions or skills — just like hermes-dev-team has one. At that point, a fresh session launched from the new project's directory will pick up those overrides. But this is only relevant after the scaffold exists. For the idea → initial scaffold step, stay in your current session.

One small workflow tip :

If you're mid-conversation about something else when the idea strikes, you can always say:

└ "Park this for a second. New topic: I want to start a new app called X. Use └ vibe-loop at /media/bob/C1/AI\_Projects/X. [description]"

Hermes will switch gears and start the scaffold. When that's kicked off (or finished), you can come back to whatever you were doing before — or open a second Hermes Chat window if you want to truly parallel-process.

The simplest invocation. Hermes uses the default model from `~/.hermes/config.yaml` ( `qwen3.5:cloud` ) and gives you a normal chat with all built-in tools (file ops, terminal, etc.):

```
cd /media/bob/C1/AI_Projects/Crispi-app
hermes chat
```

Then type and hit enter. Use `/help` inside the session for built-in commands. Use `Ctrl+D` or `/exit` to leave.

## Chat with a specific model

```
hermes chat -m qwen3.5:cloud          # explicit cloud model
hermes chat -m qwen3-coder-next:cloud # coder-tuned variant (faster on tool calls)
# hermes chat -m hermes3:8b           # local on P40s (uncomment in config first)
```

## Chat with the dev-team skills loaded (interactive vibe-loop)

This is the **interactive equivalent** of the autonomous overnight run. Hermes loads the vibe-loop skill, but instead of running the pipeline silently, it waits for you to describe what you want and then runs phases interactively. You'll be prompted to confirm phase transitions (unless you also pass `--yolo` ).

```
cd /media/bob/C1/AI_Projects/Crispi-app
hermes chat -s dev-team/vibe-loop
```

Then describe the task in plain English, e.g.:

- `""What's the status of Crispi-app-evr? Read the story spec and tell me what's left.""`
- `""Run quinn-review on the current branch. Don't write any code, just review.""`
- `""Continue Crispi-app-evr. Start at tdd. Ask me before kicking off Phase 10.""`

This is the right mode when:

- You want to **watch hermes work** in real time and steer it
- You want to **start a phase, then stop** (e.g., generate the test files but stop before implementation)
- You want to **debug a failing pipeline run** by inspecting state interactively
- You want to **try a one-off task** without committing to the full pipeline (e.g., just run quinn-review)

## Chat with multiple skills loaded

You can preload multiple skills with comma-separated `-s` or by repeating the flag:

```
hermes chat -s dev-team/vibe-loop,dev-team/health-fix
hermes chat -s dev-team/vibe-loop -s dev-team/stack-detect
```

Useful combinations for Crispi:

| What you want                          | Skills to load                 |
|----------------------------------------|--------------------------------|
| Full dev pipeline (most cases)         | dev-team/vibe-loop             |
| Refresh AGENTS.md stack info           | dev-team/stack-detect          |
| Fix lint/TS/build errors progressively | dev-team/health-fix            |
| Debug a failed Pi subagent             | dev-team/escalation-handler    |
| Pick a model tier for a story          | dev-team/model-tier-classifier |

## Resume a previous chat session

Hermes auto-persists every session to `~/hermes/sessions/session_<timestamp>_<id>.json` . You can resume any of them:

```
# Resume the most recent session (any project, any skill)
hermes chat --continue
hermes chat -c      # short form

# Resume a specific session by ID (the ID is shown when the session ends)
hermes chat --resume 20260411_094118_9fb59b
hermes chat -r 20260411_094118_9fb59b

# Resume the most recent session by name (if it has one)
hermes chat -c "evr-redesign"
```

This is **gold for debugging failed runs** — you can resume the dead session, inspect what hermes was thinking, and ask it follow-up questions about why it stopped.

To list available sessions:

```
ls -lat ~/.hermes/sessions/session_*.json | head -10
# Then pick a session_id from the filename
```

## Other useful flags for interactive mode

| Flag                         | What it does                                                                          | When to use                                                                              |
|------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>-v / --verbose</code>  | Verbose output (show reasoning, tool args in full)                                    | Debugging an agent that's making weird decisions                                         |
| <code>-Q / --quiet</code>    | Suppress banner/spinner/tool previews. Only final response.                           | Piping output to another tool, or for clean transcripts                                  |
| <code>--worktree / -w</code> | Run in an isolated git worktree                                                       | Testing risky changes in parallel without touching the main checkout                     |
| <code>--checkpoints</code>   | Filesystem checkpoints before destructive ops (use <code>/rollback</code> to restore) | When you want a safety net for <code>write_file</code> and <code>terminal</code> actions |
| <code>--max-turns N</code>   | Cap tool-calling iterations per turn (default 90 from config)                         | Debugging runaway loops                                                                  |
| <code>--yolo</code>          | Skip dangerous-command approvals                                                      | Trusted unattended runs (combine with <code>setsid -f</code> for overnight)              |
| <code>--source tool</code>   | Tag session as "tool" so it doesn't appear in normal session lists                    | Hidden background helpers                                                                |

## Interactive vs autonomous — quick decision tree

```
Do you know exactly what you want and trust hermes to run unattended?
└─ YES → setsid -f hermes chat -s dev-team/vibe-loop --yolo -q '...' (overnight pattern)
└─ NO  → hermes chat -s dev-team/vibe-loop (interactive, you steer)

Are you debugging a failed run?
└─ hermes chat -c                (resume most recent)
   hermes chat -r <session_id>   (resume specific)

Are you just asking hermes a question?
└─ hermes chat                  (no skill, default model)

Are you doing risky surgery in parallel?
└─ hermes chat -s dev-team/vibe-loop --worktree --checkpoints
```

---

## Watching it Live

### Tail the log file in another terminal:

```
tail -f ~/.hermes/logs/overnight-evr-*.log
```

### See what tools hermes is calling right now:

```
tail -f ~/.hermes/logs/agent.log | grep -E "Vision|Auxiliary|run_agent"
```

### See Ollama API activity (model calls):

```
journalctl _PID=$(pgrep -x ollama) -f | grep "POST.*chat/completions"
```

### Confirm the process is still alive:

```
ps -ef | grep "hermes chat" | grep -v grep
```

The PPID column should show `1` (init) — that confirms it's properly detached. If the PPID is anything else, it's still parented to a shell that could die and take it with it.

### Watch beads activity (issues being created/updated):

```
watch -n 30 'bd list --status=in_progress; echo; bd list --status=open | tail -10'
```

Or just check periodically:

```
bd show Crispi-app-evr    # see status + notes  
git log --oneline -10     # see commits being made
```

---

## Post-Run Inspection (When You Wake Up)

### 1. Did it finish or did it die?

```
ps -ef | grep "hermes chat" | grep -v grep
```

- **No output** = process exited (could be success OR crash, check log)
- **Process still running** = still going. Don't kill it. Check `tail` of log to see what phase it's on.

### 2. Read the tail of the log

```
tail -100 ~/.hermes/logs/overnight-evr-*.log
```

Look for:

- `Phase 13 complete` or `Vibe Loop COMPLETE` → success
- `quinn-review findings` → adversarial review ran (mandatory hard gate)
- Python tracebacks → crash. Save the log and share with Sally/whoever is debugging.
- Empty tail / abrupt cutoff → SIGHUP-style death (rare with `setsid`, but possible if system rebooted)

### 3. Check what got committed

```
git log --oneline --since="12 hours ago"  
git status
```

- Look for commits with story IDs in the message
- Working tree should be clean if the run completed
- Untracked files = run died before commit

## 4. Check beads state

```
bd show Crispi-app-evr
bd list --status=in_progress    # should be empty if run completed
bd list --status=closed --since="12 hours ago"
```

## 5. Check pipeline log for phase transitions

```
cat _output/pipeline.log
```

Vibe-loop appends one line per phase transition. If it stops mid-list, that's where it died.

# Recovery From a Failed Run

## Vibe-loop auto-detects what's complete

You do NOT need to figure out which phase to start at — vibe-loop has built-in **Redundancy Detection**. In `--yolo` mode, it automatically uses existing artifacts:

| If this exists                                          | Vibe-loop                                          |
|---------------------------------------------------------|----------------------------------------------------|
| <code>_output/feature-spec.md</code>                    | Skips Phase 4 (prd)                                |
| <code>_output/architecture.md</code>                    | Skips Phase 5 (architecture)                       |
| <code>_output/epics.md</code>                           | Skips Phase 6 (epics)                              |
| <code>docs/stories/*.md</code> (matching)               | Skips Phase 7a (story-specs)                       |
| Test files in <code>__tests__</code> / (matching story) | Skips Phase 7b (tdd)                               |
| Beads issues with matching label                        | Skips Phase 8 (beads-filing)                       |
| Discovery commit in <code>git log</code>                | Skips Phase 9 (checkpoint)                         |
| <code>bd ready</code> returns issues                    | Auto-resumes Phase 10 (dev) from where it left off |

So **the same launch command works for re-kicking a failed run**. Just run it again. Vibe-loop will pick up at the first incomplete phase.

The only time you'd manually specify a start phase is if you want to **FORCE** a specific phase to run (e.g., re-run Phase 2 immersion to refresh the AGENTS.md project rules block):

```
hermes chat -s dev-team/vibe-loop --yolo -m qwen3.5:cloud \
  -q "Continue Crispi-app-evr. Force re-run immersion (Phase 2) to refresh AGENTS.md rules block."
```



# Hermes Context Compaction (Reference)

`qwen3.5:cloud` has a **256K token context window**. Hermes auto-compresses when conversation hits 50% (131K tokens), targeting a 5x reduction while preserving the last 20 messages. Compression uses the same model as the worker ( `summary_model: qwen3.5:cloud` in `~/.hermes/config.yaml` ).

For one vibe-loop pipeline run on a feature-sized task, you almost certainly will NOT hit the compression threshold. The pipeline writes artifacts to disk between phases instead of keeping file contents in conversation history.

If you DO see compression happening in the log ( `context compression started: session=...` ), it's a sign that the conversation is unusually long — likely the dev phase is iterating many times on test failures. That's not necessarily a bug, but if it compresses 3+ times in one phase, the model is probably stuck in a loop and you should kill the run and investigate.

**Memories survive compression.** Hermes calls `flush_memories()` BEFORE compressing, so anything the agent decided to remember mid-thinking gets saved to `~/.hermes/memories/` first.

## What Pi Sees (Why AGENTS.md Matters)

Pi (the dev coder subagent in Phase 10) does NOT read this guide, vibe-loop's SKILL.md, or `_bmad-output/project-context.md` . Pi gets project knowledge through TWO channels:

- 1. **AGENTS.md** — Pi reads this on every invocation. The `<!-- BEGIN PROJECT RULES -->` block (currently injected manually, normally injected by Phase 2 immersion) contains critical don'ts and security rules.
- 2. **Story file Dev Notes** — Pi reads the story spec for the issue it's working on. The `## Coding Rules` section in the story file (auto-injected by Phase 7a story-specs) contains task-specific file paths and patterns to reuse.

If `AGENTS.md` is missing the `<!-- BEGIN PROJECT RULES -->` block, Pi will work with channel #2 only. To verify before launch:

```
grep -c "BEGIN PROJECT RULES" AGENTS.md # should return 1, not 0
```

## Failure Modes Seen So Far

| Date             | Symptom                                        | Cause                                                                                              | Fix                                                                                                               |
|------------------|------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 2026-04-10 17:04 | Run died after Phase 7a, only wrote story spec | Claude Code parent died → SIGHUP cascaded → hermes had no SIGHUP handler → process killed mid-call | Use <code>setsid -f</code> to detach. Tracked as a hermes-agent upstream improvement (file SIGHUP handler patch). |

(Add new entries here as you debug future runs.)

---

## Quick Reference Card

```
# Launch (always run from a real terminal, NOT Claude Code bash tool)
LOG=~/.hermes/logs/overnight-$(date +%s).log
setsid -f bash -c "cd /media/bob/C1/AI_Projects/Crispi-app && hermes chat -s dev-
team/vibe-loop --yolo -m qwen3.5:cloud -q 'YOUR PROMPT HERE' > $LOG 2>&1"
echo "Log: $LOG"

# Watch live (in another terminal)
tail -f $LOG

# Verify detached
ps -ef | grep "hermes chat" | grep -v grep
# PPID column should be 1

# Check beads / git after wake-up
bd show <issue-id>
git log --oneline --since="12 hours ago"
cat _output/pipeline.log
```